



IMPLÉMENTEZ UN MODÈLE DE SCORING

ANTONINE LAROZE-CERVETTI

OBJECTIFS

- Construire un modèle de scoring qui donne une prédiction sur la probabilité de faillite d'un client de façon automatique.
- Analyser les features importances
- Mettre en production le modèle de scoring de prédiction à l'aide d'une API et réaliser une interface de test (via Streamlit [ici](#))
- Analyser le data drift

ENJEUX MÉTIER

```
TARGET
0.0    282682
1.0    24825
Name: count, dtype: int64
```

Déséquilibre de la variable cible TARGET

- 0 = bon client, qui ne fait pas défaut
- 1 = mauvais client, fait défaut

- Vrai positif : mauvais client qui fait effectivement défaut
- Vrai négatif: bon client qui ne fait effectivement pas défaut
- Faux positif: bon client qui se voit son crédit refusé à tort
- Faux négatif: mauvais client qui se voit accepter son credit à tort

Pour l'entreprise, un faux négatif coûte 10x plus cher qu'un faux positif

PRÉSENTATION DES DONNÉES

8 tables :

- application_{train|test}.csv : Données sur chaque demande de prêt (train contient la cible, test non).
- bureau.csv : Crédits antérieurs des clients auprès d'autres institutions.
- bureau_balance.csv : Historique mensuel des crédits référencés dans bureau.csv.
- POS_CASH_balance.csv : Historique mensuel des anciens crédits à la consommation chez Home Credit.
- credit_card_balance.csv : Historique mensuel des anciennes cartes de crédit chez Home Credit.
- previous_application.csv : Anciennes demandes de prêt chez Home Credit.
- installments_payments.csv : Historique de paiement des anciens crédits Home Credit.



➔ Travail d'aggrégation (table avec ligne par prêt/ligne par client), création de nouvelles variables et travail de jointure pour avoir une table finale.

NETTOYAGE DES DONNÉES

- VALEURS MANQUANTES:
 - « Unknown » pour les variables qualitatives
 - Valeurs médianes pour les variables quantitatives
- VALEURS ABERRANTES:
 - Variable « nombre d'enfant » ramenée à 1, 2, 3 et plus;
 - Genre limité à féminin et masculin.

VARIABLES EXPLICATIVES

Après filtre: 74 features et plus de 300,000 lignes

- **Informations personnelles:** Age, genre, propriétaire, nombres d'enfants, statut familial, région etc.
- **Revenus et crédits demandés:** revenu, montant du crédit demandé, durée du crédit, taux de paiement, annuité etc.
- **Historique bancaires et crédits:** scores de risque externes fournis par d'autres institutions bancaires, montant des crédits moyens demandés, durée moyenne de retards de paiement, défaut moyen, balance moyenne sur les cartes de crédit etc.
- **Informations professionnelles:** le type de revenu, l'emploi occupé, secteur d'emploi, niveau d'éducation



FEATURE ENGINEERING

Aggrégation de features :

- Pour les tables qui concernent l'historique de demandes de crédits => aggrégation car chaque demande actuelle de crédit (client) peut avoir plusieurs historiques de crédits
- Principalement des moyennes et maximums: retard moyen de paiement, montant moyen des crédits accordés, etc.

Création de variables:

- Rapport entre revenu annuel et crédit demandé = mesure la capacité de remboursement d'un client via ses revenus
- Revenu par membre du foyer
- Rapport entre l'annuité et le revenu total = indique le poids des mensualités par rapport au revenu
- Taux de remboursement (annuité sur crédit total): Évalue la rapidité à laquelle un client rembourse son crédit

— GESTION DU DÉSEQUILIBRE

Déséquilibre des classes

- 92 % des clients remboursent leur crédit.
- Le modèle apprend à prédire systématiquement la classe majoritaire
- Accuracy biaisée (~92 %) mais inefficace pour détecter les clients à risque.
- Coût des erreurs: Les faux négatifs (mauvais payeurs non détectés) ont un impact financier élevé.
- Accorder un crédit à un client défaillant coûte plus cher que refuser un bon payeur.

Méthodologie

- **SMOTE** (Synthetic Minority Over-sampling Technique): Génère des exemples synthétiques pour équilibrer les classes grâce aux k plus proches voisins.
- Appliqué uniquement sur les données d'entraînement
- **Class_weight**: paramètre de modèle qui donne plus de poids à la classe minoritaire
- **Optimisation et évaluation**: BayesSearchCV
- **Optimisation du seuil de décision** selon les enjeux métier.
- **Métriques adaptées** : ROC AUC, précision, rappel, création d'un score métier



MODÉLISATION

- **Régression logistique:** Modèle linéaire qui prédit la probabilité d'appartenir à une classe en utilisant une fonction logistique.
 - ➔ Modèle linéaire, simple et facilement interprétable
- **Lightgbm:** Modèle de gradient boosting optimisé pour la performance, basé sur des arbres de décision construits de manière itérative (apprend de ses erreurs).
 - ➔ Modèle d'ensemble, performant, rapide et gère mieux les données déséquilibrées
- **RandomForest:** Ensemble d'arbres de décision bagging (agrégation aléatoire). Il réduit le surapprentissage en combinant plusieurs arbres peu corrélés.
 - ➔ Modèle d'ensemble robuste aux valeurs aberrantes

MODÈLES TESTÉS

- **Dummy classifier:** modèle de base qui prédit selon une stratégie simple. Il n'apprend rien des données.
 - ➔ Sert uniquement de référence de performance minimale.
- **Naives Bayes (Gaussian):** Classificateur probabiliste basé sur le théorème de Bayes, en supposant une indépendance forte entre les variables. La version gaussienne suppose que les variables suivent une distribution normale
 - ➔ Modèle génératif probabiliste. Rapide mais parfois moins réaliste.

CONFIGURATION ET MÉTRIQUES

- Création des jeux de données (train 70%, test (val) 30%)

GridSearchCV vs BayesSearchCV:

- GridSearchCV explore exhaustivement toutes les combinaisons d'hyperparamètres sur une grille définie. **Méthode simple mais coûteuse en temps de calcul,**
- Bayes : modélise la performance du modèle en fonction des hyperparamètres. Choisit intelligemment les combinaisons à tester en apprenant des essais précédents.
- Plus rapide et souvent plus performant

Métriques

- **F1** (moyenne harmonique entre précision et rappel),
- **Précision** (proportion de prédictions positives qui sont correctes)
Parmi tous les clients que le modèle a prédits comme "mauvais payeurs", combien le sont réellement
- **Rappel** (proportion de vrais positifs détectés parmi tous les vrais cas positifs): Parmi tous les vrais mauvais payeurs, combien le modèle a réussi à identifier correctement
- **ROC AUC** (Aire sous la courbe ROC): Mesure la capacité globale du modèle à différencier un mauvais payeur d'un bon payeur, quel que soit le seuil
- **Accuracy:** Proportion de prédictions correctes toutes classes confondues
- **Score métier:** $10 \times \text{FN} + 1 \times \text{FP}$
- Recherche d'un seuil de classification pour optimiser le score métier

RÉSULTATS

Modèle	Timer	Seuil	Precision	Recall	Accuracy	F1	Coût	ROC AUC
Dummy (s)	>1s	0,50	0,48	0,49	0,50	0,08	53994	0,49
Dummy (mf)	>1s	0,50	0	0	0,91	0	49650	0,50
Naive Bayes	>1s	0,50	0,13	0,54	0,68	0,21	39900	0,65
	>1s	0,65	0,14	0,52	0,71	0,22	39656	0,66
Logistic Regression	45 min	0,50	0,16	0,67	0,70	0,26	33343	0,74
	45 min	0,50	0,16	0,67	0,70	0,26	33343	0,74
Random Forest	> 2h	0,50	0,57	0	0,92	0	49502	0,73
	> 2h	0,15	0,17	0,60	0,75	0,26	34278	0,73
LightGBM	11 min	0,50	0,57	0,03	0,91	0,06	48143	0,77
	11 min	0,10	0,19	0,64	0,75	0,30	31227	0,77

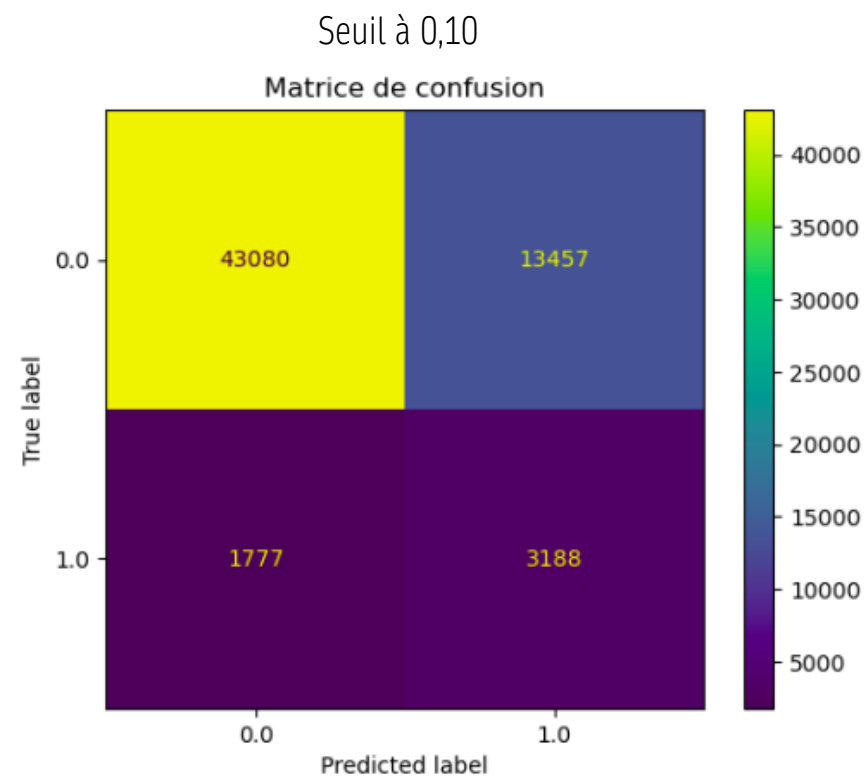
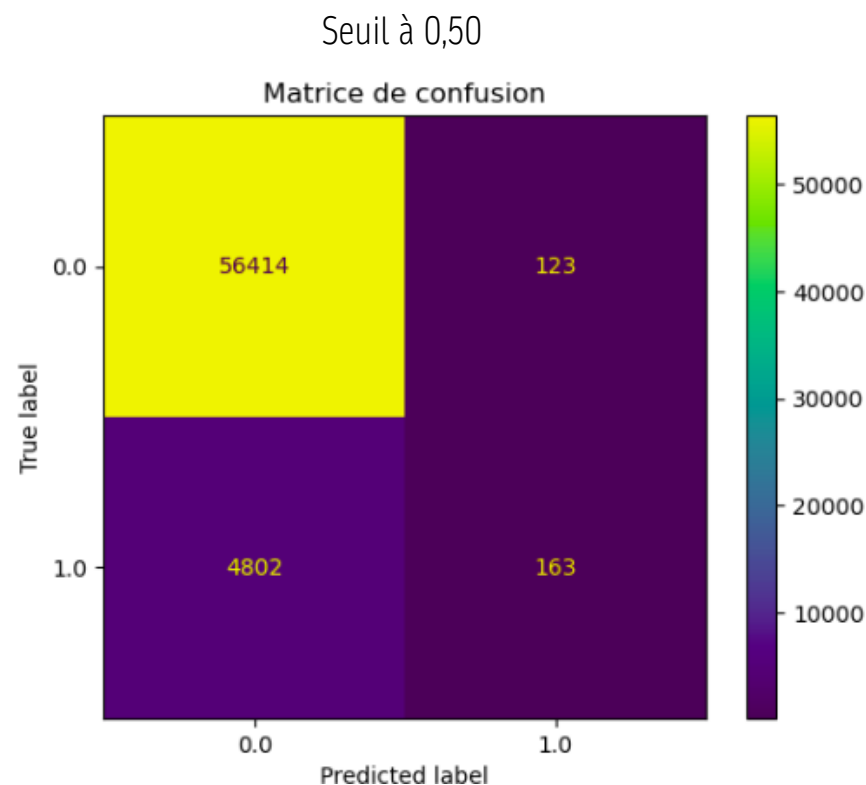
 *Nota Bene* : Les métriques présentées sont issues des *pipelines finaux sans fuite de données* ; seule la *recherche d'hyperparamètres* a été effectuée avec *SMOTE hors validation croisée*, ce qui peut introduire un léger optimisme sur les paramètres choisis,

SYNTHÈSE

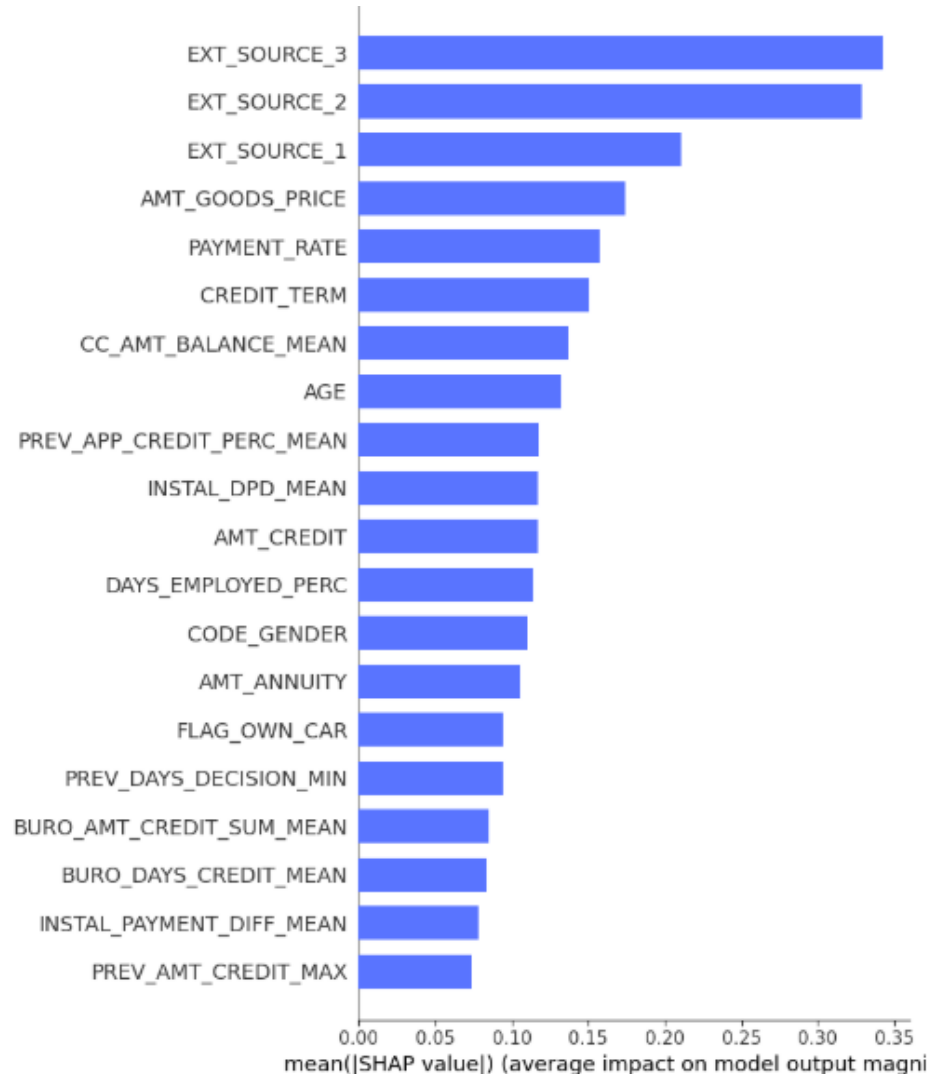
Par défaut, le modèle utilise un **seuil de 0,50**, ce qui signifie qu'un client est classé comme "mauvais payeur" uniquement si sa probabilité dépasse 50 %.

Dans notre cas, le **seuil optimisé chute à 0,10**. Le modèle est plus strict : dès qu'un client présente plus de 10 % de risque de défaut, le crédit est refusé.

Cette stratégie permet de **réduire significativement les faux négatifs** et donc de **limiter les pertes financières**.



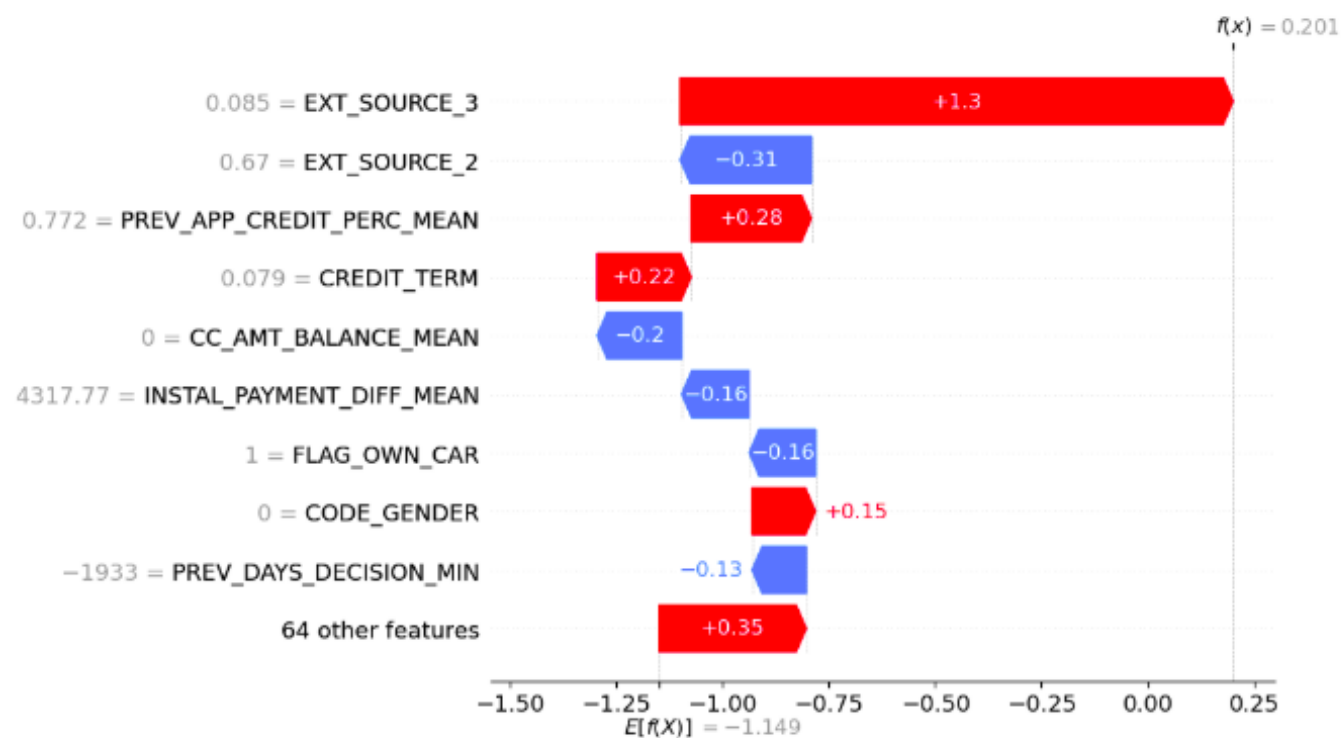
FEATURE IMPORTANCE



- **SHAP:** théorie des jeux de Shapley qui attribue à chaque feature une contribution moyenne pondérée sur toutes les prédictions.
- L'analyse globale montre que les variables ayant le plus fort impact sur les décisions du modèle sont, en priorité, **les scores externes** qui capturent la solvabilité estimée par des institutions bancaires,
- Viennent ensuite des **variables financières clés** telles que le montant du bien acheté avec le crédit, le rythme de remboursement (capacité) et la durée de l'engagement.
- D'autres indicateurs importants incluent la **situation financière historique** et des **caractéristiques personnelles**

FEATURE IMPORTANCE

Client sélectionné : 458



- Pour ce client, la variable EXT_SOURCE_3 contribue fortement à augmenter le risque de défaut (valeur très faible 0,08 = profil risqué),
- À l'inverse, d'autres variables comme EXT_SOURCE_2 (score externe plus élevé 0,67) et la régularité des paiements passés (INSTAL_PAYMENT_DIFF_MEAN) viennent réduire ce risque.
- La probabilité de faire défaut pour ce client est de 0,20 donc > au seuil de 0,10. Son crédit sera donc refusé.

SUIVI ET TRACABILITÉ

MLflow est une plateforme qui permet de **gérer, versionner, et déployer** des modèles de Machine Learning.

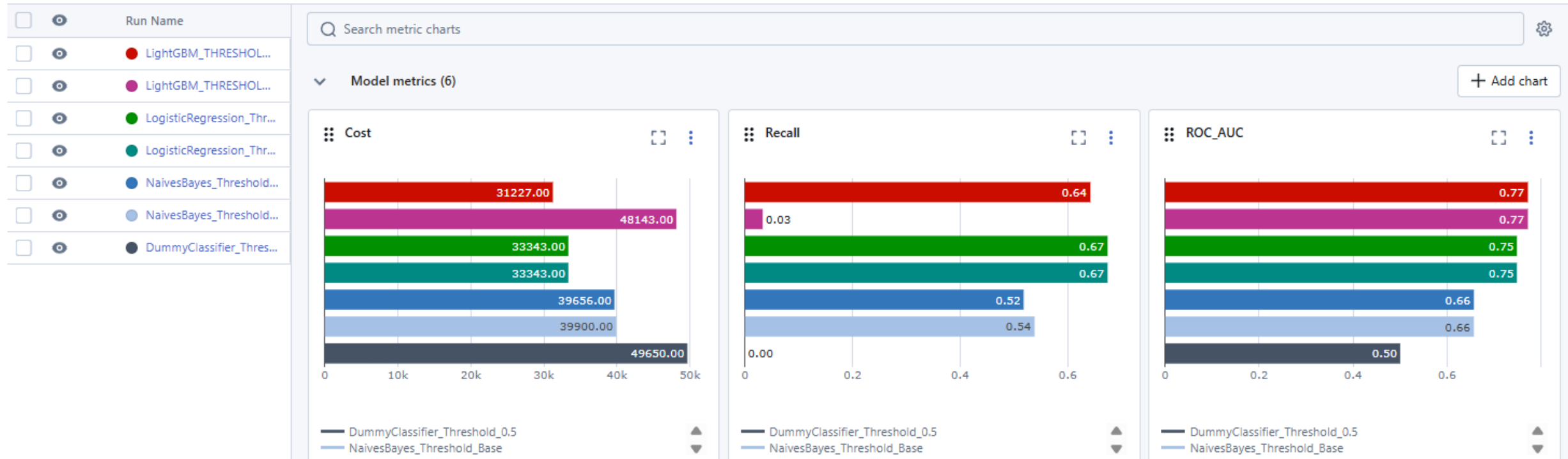
Model Registry		
Name 	Latest version	Aliased versions
DummyClassifier	Version 1	
LightGBM	Version 2	@ final : Version 2
LogisticRegression	Version 2	
NaivesBayes	Version 2	

Expérimentations (attention le modèle RandomForest n'a pas pu être versionné car trop lourd)

Runs										
Experimental										
Traces										
metrics.rmse < 1 and params.model = "tree"										
Time created										
State: Active										
Datasets										
Sort: Created										
Columns										
Group by										
Metrics										
☐	Run Name	Created 	Duration	Models	Accuracy	Cost	F1_Score	Precision	ROC_AUC	Recall
☐	LightGBM_THRESHOLD_Optimal	5 days ago	10.7s	LightGBM v2 +1	0.75230073...	31227	0.29504858...	0.19152898...	0.77175305...	0.64209466...
☐	LightGBM_THRESHOLD_050	5 days ago	11.0s	LightGBM v1 +1	0.91992130...	48143	0.06208341...	0.56993006...	0.77175305...	0.03282980...
☐	LogisticRegression_Threshold_Optimal	5 days ago	9.5s	LogisticRegression ... +1	0.69462781...	33343	0.26276741...	0.16318868...	0.74838872...	0.67411883...
☐	LogisticRegression_Threshold_050	5 days ago	10.6s	LogisticRegression ... +1	0.69462781...	33343	0.26276741...	0.16318868...	0.74838872...	0.67411883...
☐	NaivesBayes_Threshold_Optimal	5 days ago	10.7s	NaivesBayes v2 +1	0.70539169...	39656	0.22112367...	0.14056181...	0.65680931...	0.51802618...
☐	NaivesBayes_Threshold_Base	5 days ago	10.5s	NaivesBayes v1 +1	0.68664433...	39900	0.21715817...	0.13600976...	0.65680931...	0.53836858...
☐	DummyClassifier_Threshold_0.5	5 days ago	22.4s	DummyClassifier v1 +1	0.91927091...	49650	0	0	0.5	0

SUIVI ET TRACABILITÉ

MLflow nous permet aussi de naviguer et comparer nos modèles en fonction des métriques. Ici on retrouve bien les métriques des modèles testés.





API

ARCHITECTURE DE L'API

- NOM : API Scoring Client

- Endpoints

- **HEAD /**

Description : Vérifie la disponibilité de l'API (healthcheck).

Réponse : 200 OK (sans corps)

- **GET /**

Description : Message d'accueil.

Réponse : {"message": "Bienvenue sur l'API de scoring client. Utilisez /predict pour prédire."}

- **GET /client/**

Description : Retourne la liste des identifiants client (SK_ID_CURR).

- **GET /client/{client_id}**

Description : Retourne les données d'un client spécifique. Réponses : 200 OK : dictionnaire de features du client sinon 404 Not Found : le client est introuvable

- **POST /predict**

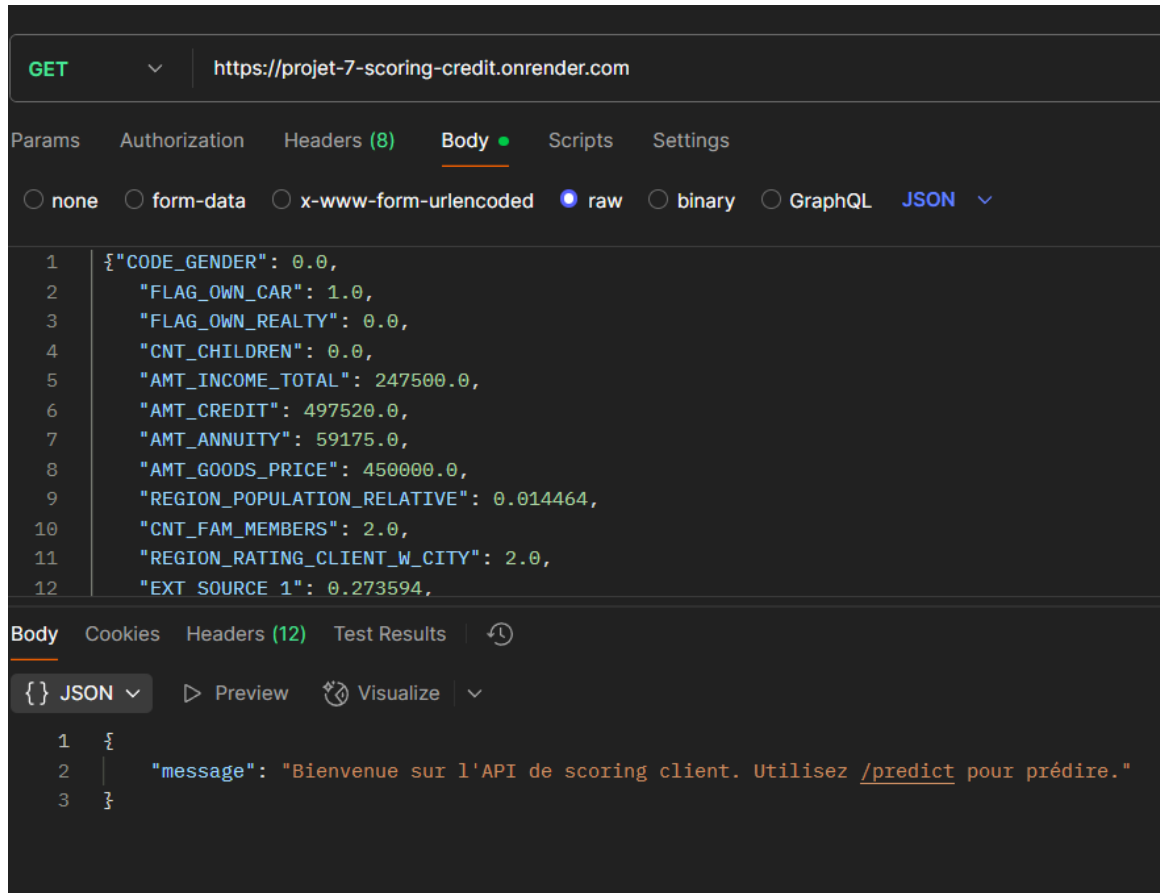
Description : Prédiction de la probabilité de défaut de paiement.

Corps de la requête : JSON selon le schéma InputData

Réponse : { "proba": 0.0, "décision": "accepté" }

Pour des raisons de capacité l'API est produite sur un sous-échantillon du dataset original (+ de 300k lignes), L'échantillon contient 5000 lignes

REQUÊTE AVEC POSTMAN



Postman interface showing a GET request to `https://projet-7-scoring-credit.onrender.com`. The request body is raw JSON, containing a list of 12 parameters. The response body is JSON, containing a message.

GET `https://projet-7-scoring-credit.onrender.com`

Params Authorization Headers (8) Body Scripts Settings

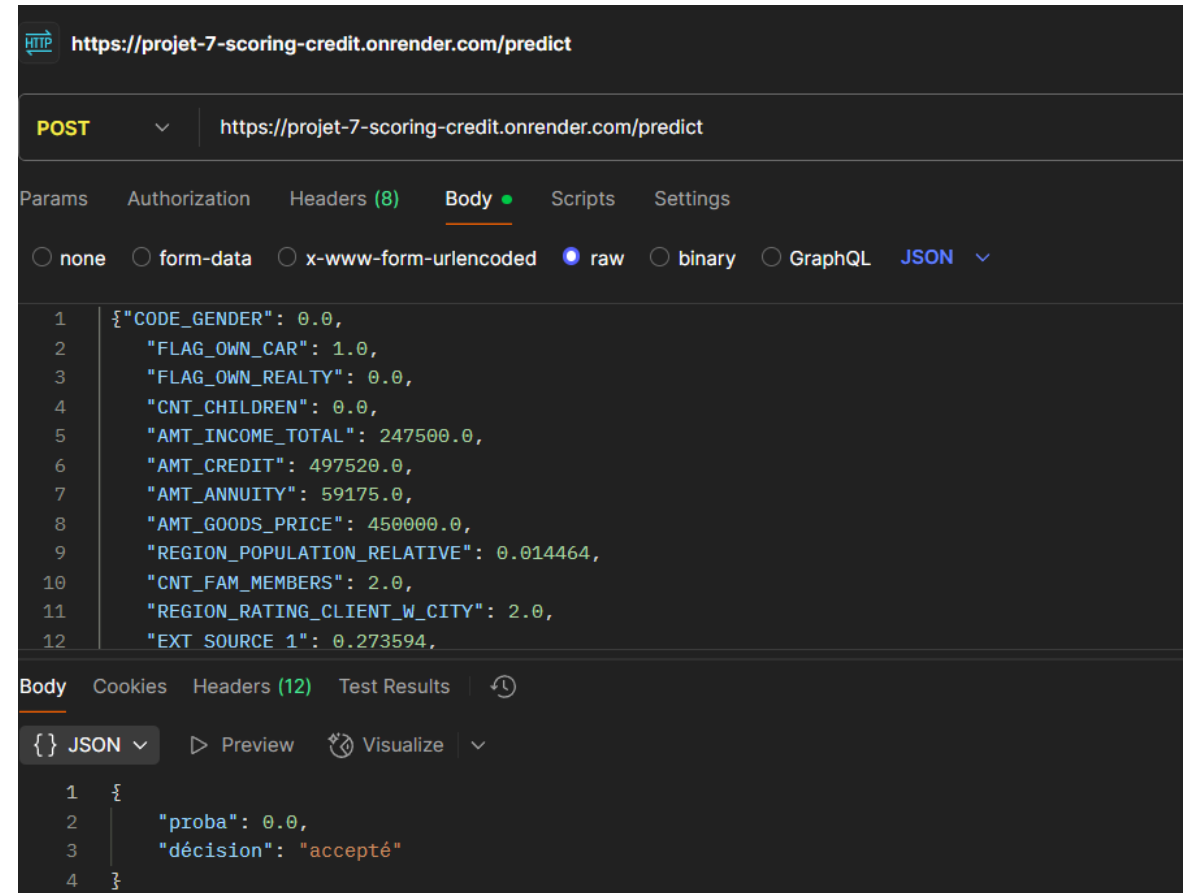
☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☒ JSON

```
1 {"CODE_GENDER": 0.0,
2   "FLAG_OWN_CAR": 1.0,
3   "FLAG_OWN_REALTY": 0.0,
4   "CNT_CHILDREN": 0.0,
5   "AMT_INCOME_TOTAL": 247500.0,
6   "AMT_CREDIT": 497520.0,
7   "AMT_ANNUITY": 59175.0,
8   "AMT_GOODS_PRICE": 450000.0,
9   "REGION_POPULATION_RELATIVE": 0.014464,
10  "CNT_FAM_MEMBERS": 2.0,
11  "REGION_RATING_CLIENT_W_CITY": 2.0,
12  "EXT_SOURCE_1": 0.273594,
```

Body Cookies Headers (12) Test Results

☒ JSON Preview Visualize

```
1 {
2   "message": "Bienvenue sur l'API de scoring client. Utilisez /predict pour prédire."
3 }
```



Postman interface showing a POST request to `https://projet-7-scoring-credit.onrender.com/predict`. The request body is raw JSON, containing a list of 12 parameters. The response body is JSON, containing a prediction result.

POST `https://projet-7-scoring-credit.onrender.com/predict`

Params Authorization Headers (8) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☒ JSON

```
1 {"CODE_GENDER": 0.0,
2   "FLAG_OWN_CAR": 1.0,
3   "FLAG_OWN_REALTY": 0.0,
4   "CNT_CHILDREN": 0.0,
5   "AMT_INCOME_TOTAL": 247500.0,
6   "AMT_CREDIT": 497520.0,
7   "AMT_ANNUITY": 59175.0,
8   "AMT_GOODS_PRICE": 450000.0,
9   "REGION_POPULATION_RELATIVE": 0.014464,
10  "CNT_FAM_MEMBERS": 2.0,
11  "REGION_RATING_CLIENT_W_CITY": 2.0,
12  "EXT_SOURCE_1": 0.273594,
```

Body Cookies Headers (12) Test Results

☒ JSON Preview Visualize

```
1 {
2   "proba": 0.0,
3   "décision": "accepté"
4 }
```

INTERFACE STREAMLIT

Application de Scoring Client

Sélectionnez un ID crédit

100021.0

Modifiez les caractéristiques du client pour simuler un scénario

Informations personnelles

Sexe

☐ Femme

☒ Homme

Possède une voiture ?

☒ Non

☐ Oui

Possède un bien immobilier ?

☐ Non

☒ Oui

Nombre d'enfants

1

Nombre de membres dans la famille

3

Âge

26

Statut familial

Informations professionnelles

Type de revenu

Working

Niveau d'éducation

Secondary / secondary special

Secteur d'activité

Industry

Profession

Labor_Work

[Prédire la probabilité de défaut](#)

Probabilité de défaut : 0.00%

✓ Crédit ACCEPTÉ (risque acceptable)

- Permet de requêter automatiquement l'API pour avoir une prédiction
- Sélection des ID de demandes de crédit qui remplissent automatiquement les features
- Possibilité de modifier les features afin de tester d'autres scénarios

LIEN API/STREAMLIT

Streamlit interagit avec l'API en plusieurs étapes :

- ❑ 1^{ère} requête à l'API pour récupérer la liste des clients (/client).
- ❑ Lorsqu'un client est sélectionné, une 2^{ème} requête est envoyée pour charger ses données complètes (/client/{client_id}).

L'utilisateur peut alors modifier certaines valeurs pour tester différents scénarios grâce à des composants interactifs :

- st.radio (choix unique),
 - st.selectbox (menu déroulant),
 - st.number_input (valeurs numériques).
-
- ❑ Une 3^{ème} requête est ensuite envoyée pour obtenir une prédiction via le bouton st.button, qui appelle l'endpoint /predict.

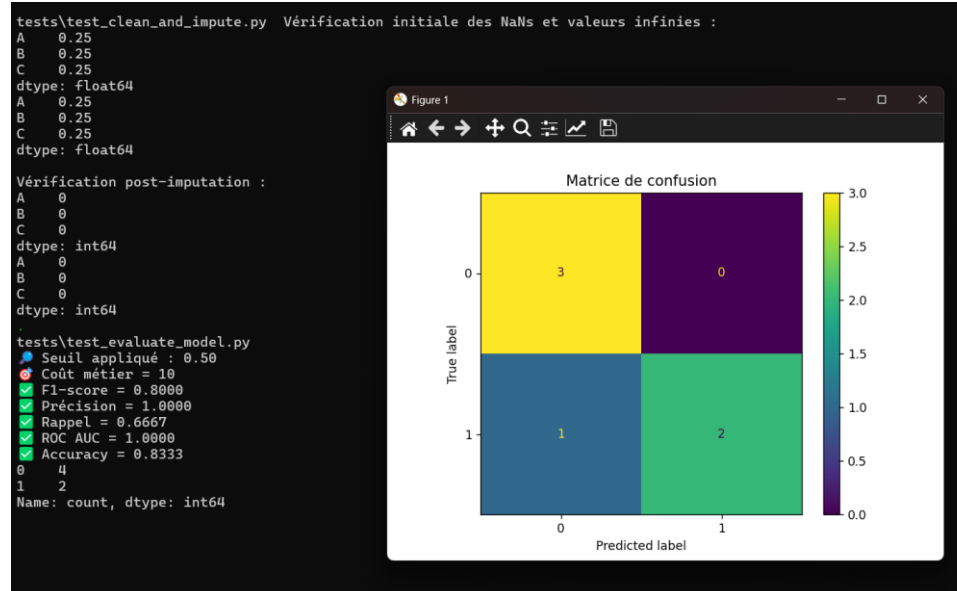
Le résultat (probabilité + décision) est affiché directement dans l'interface.

PYTEST – GITHUB ACTIONS

Pytest permet d'écrire des tests simples, lisibles et puissants pour garantir la fiabilité du code.

GitHub Actions (outil d'automatisation) va permettre d'automatiser ces tests à chaque évènements (push, pull etc.)

1. **test_clean_and_impute**: tester si les valeurs manquantes et les valeurs infinies sont bien remplacées
2. **test_evaluate_model**: tester si les bonnes métriques sont calculées et si la matrice de confusion s'affiche bien
3. **test_find_optimal_threshold**: tester si le seuil optimal se calcule bien
4. **test_split_and_scale** : tester si les données sont bien découpées en ensembles d'entraînement et de validation (test), que la normalisation est effectuée correctement et que les dimensions des matrices retournées sont correctes.



PYTEST – GITHUB ACTIONS

Processus Git & GitHub

- `git init` → Initialiser un dépôt Git local
 - `git add .` → Ajouter tous les fichiers au suivi
 - `git commit -m "message"` → Sauvegarder les modifications
 - `git remote add origin <URL>` → Lier au dépôt GitHub
 - `git push -u origin main` → Envoyer le code sur GitHub
- *git pull* → Récupérer les dernières modifications
 - *git fetch* → Vérifier les mises à jour distantes sans les appliquer

```
tests/test_clean_and_impute.py::test_clean_and_impute_removes_nans_and_infs PASSED [ 25%]
tests/test_evaluate_model.py::test_evaluate_model PASSED [ 50%]
tests/test_evaluate_model.py::test_find_optimal_threshold PASSED [ 75%]
tests/test_split_and_scale.py::test_split_and_scale PASSED [100%]

===== 4 passed in 1.65s =====
```

✓ Ajout du README pour API #10

Summary

Jobs

- ✓ tests
- ✓ deploy

Run details

- Usage
- Workflow file

Triggered via push 1 hour ago

Anto1293 pushed 2635853 master

Status: Success

Total duration: 1m 22s

Artifacts: -

cicd.yml

on: push

tests 1m 6s → deploy 7s

DATA DRIFT

- Phénomène lorsque les données deviennent différentes que celle utilisées pour l'entraînement du modèle
- Utilisation de la bibliothèque Evidently qui permet de détecter et visualiser le data drift, les changements de distribution
- Utilisation de la table application_test pour les nouvelles données
- Les drifts peuvent venir : de changements économiques ou sociaux, de nouvelles politiques internes de la banque ou de l'évolution du comportement des clients
- Pas de data drift car seulement 9 features ont changées par rapport aux données de référence

Dataset Drift

Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5

74

Columns

9

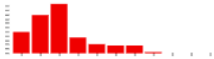
Drifted Columns

0.122

Share of Drifted Columns

DATA DRIFT

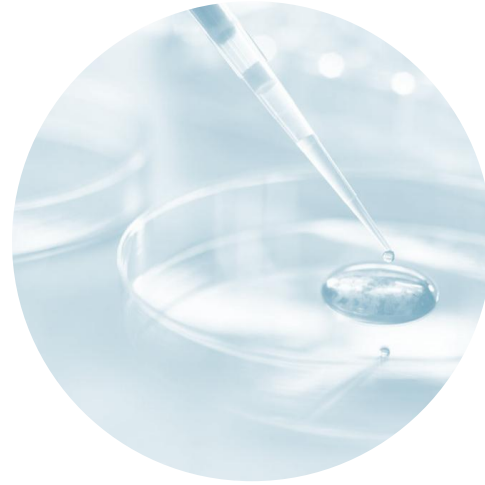
- Evidently utilise la Wasserstein distance pour comparer les distributions. Elle mesure combien la "forme" de la variable a changé
- Montant du crédit demandé
- Solde moyen des cartes de crédit (habitude de consommation change)
- Si le crédit est de type revolving ou non plus de crédits revolving en période de crise par exemple
- Durée de remboursement du crédit: Le profil des prêts peut évoluer (ex : + de prêts courts en période d'instabilité économique).
- Capacité de remboursement (part du crédit total remboursée)
- Prix du bien pour lequel le crédit est demandé (contexte économique, l'inflation, le marché immobilier)

Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
> CREDIT_TERM	num			Detected	Wasserstein distance (normed)	0.525942
> PAYMENT_RATE	num			Detected	Wasserstein distance (normed)	0.525942
> AMT_GOODS_PRICE	num			Detected	Wasserstein distance (normed)	0.231074
> AMT_CREDIT	num			Detected	Wasserstein distance (normed)	0.22839
> CC_AMT_BALANCE_MEAN	num			Detected	Wasserstein distance (normed)	0.152304
> NAME_CONTRACT_TYPE_Revolving loans	num			Detected	Jensen-Shannon distance	0.147537

CONCLUSION

- Pour contrer le déséquilibre de classes plusieurs approches ont été testées: SMOTE, le paramètre `class_weighted`, les métriques adaptées ainsi que le modèle LightGBM
- C'est le modèle LightGBM qui s'est montré le plus performant et le plus équilibré tout en réduisant le coût pour l'institution bancaire
- Les variables ayant le plus d'impact sur la prédiction sont les scores externes, les variables financières ainsi que l'historique de crédit
- Enfin, une fois le modèle déployé, l'analyse a montré peu de dérive des données, sauf pour 9 variables sensibles au contexte économique ou aux habitudes de consommation





MERCI



ANTONINE LAROZE-CERVETTI